

INTRODUCTION TO PROGRAMMING LANGUAGE

1. How Does a Computer Work?

A computer is an electronic machine. It performs four basic operations: Input, Processing, Storage, and Output.

Computer Workflow:

Input → Processing → Storage → Output

Example:

You type: 5 + 10

↓

Computer processes the operation

↓

Answer = 15

Think of a computer as a very fast but obedient worker that only follows instructions.

2. What Language Does a Computer Understand?

Computers understand only Binary Language (0 and 1).

0 = OFF

1 = ON

Example:

101010101100110011110000

The computer directly understands this. But humans find it difficult to understand.

3. How Did Humans Program Computers Earlier?

In the early days, programmers wrote instructions directly in Machine Language.

Example:

101100001110000100011100

Problems:

- Difficult to read
- Difficult to write
- Easy to make mistakes

- Hard to debug
- Time-consuming

4. Assembly Language (Low-Level Language)

To solve this problem, Assembly Language was introduced.

Instead of writing:

10110000

Programmers could write:

```
MOV A, B
ADD A, C
```

These words are called **Mnemonics**.

Examples:

```
MOV = Move
ADD = Add
SUB = Subtract
MUL = Multiply
```

5. Problems with Assembly Language

Assembly Language is machine dependent.

Programs written for one processor may not run on another.

Problems:

- Hard to learn
- Difficult for large programs
- Time-consuming
- Different processors use different instructions

So programmers wanted something closer to human language.

6. Introduction of High-Level Languages

High-level languages were created .

Instead of writing

```
MOV A , B
```

```
ADD A , C
```

You can write

```
sum = a + b;
```

Much easier !!

Human can understand

Computers cannot understand this directly, so a translator is needed.

7. Who Translates High-Level Language?

A special software called a **Compiler**.

Compiler Translation Process:

Source Code → Compiler → Machine Code → Execution → Output

Example :

You write :-

```
printf("Hello");
```

Compiler converts into machine language .

Then the computer executes it

8. Why Was C Language Introduced?

In the In the 1970s, programmers mainly used:

- Assembly Language
- B Language

Assembly was difficult.

B language lacked many features.

So in 1972, a computer scientist named:

Dennis Ritchie developed **C Language** at Bell Labs

Goals:

- Easier than Assembly
- Faster execution
- Hardware interaction
- Operating System

9. Why Is C Called a Middle-Level Language?

C combines features of:

High-Level Language

Easy syntax:

a = b + c;

And Low-Level Language

Can access memory directly using pointers.

Can interact closely with hardware.

Therefore:

Assembly $\leftarrow$ C $\rightarrow$ High-Level Languages

C sits between both worlds.

That's why it is often called a **Middle-Level Language**.

10. Features of C Language

- Simple
- Fast
- Portable
- Structured
- Rich Library Functions
- Supports Pointers
- Efficient Memory Usage
- Foundation for Other Languages

11. How a C Program Works

Programmer Writes C Code

↓

Compiler Translates Code

↓

Machine Code Generated

↓

Computer Executes Code

↓

Output Displayed

History of C Language (Complete Story)

Step 1: Machine Language (The Beginning)

In the earliest days of computing, computers only understood binary code.

Syntax: Consisted entirely of 0s and 1s (e.g., 10101010, 11001100).

Drawbacks: Highly difficult to write, nearly impossible to understand, highly error-prone, and incredibly time-consuming.

Step 2: Assembly Language

To bridge the gap between humans and hardware, Assembly Language was introduced.

Syntax: Replaced binary streams with short text mnemonics like ADD, SUB, and MOV.

The Critical Problem: It was completely **non-portable**. A program written for one specific processor architecture would not work on another.

Step 3: BCPL Language (1966)

Full Name: Basic Combined Programming Language

Developer: Martin Richards

Purpose: Designed as one of the first high-level system programming languages to simplify writing operating systems and compilers compared to Assembly.

Limitation: It was still too large and complex to run efficiently on the smaller, resource-constrained systems of that era.

Step 4: B Language (1969)

Developer: Ken Thompson (Bell Labs)

Context: Thompson was actively developing the **Unix Operating System** and needed a stripped-down, simpler version of BCPL that could fit on the hardware of the time.

Limitations of B: * It was a **typeless language** (it lacked distinct data types like int, char, or float).

It was less efficient and could not fully exploit the capabilities of newer, incoming computer hardware.

Step 5: C Language

Developer: Dennis Ritchie

Year / Location: 1972 at AT&T's Bell Labs

Why the name "C"? It was simply the direct successor to the "B" language.

Assembly ---> BCPL ---> B ---> C

Why C Was Created & Its Impact on Unix

Dennis Ritchie designed C specifically to **rewrite the Unix Operating System**.

Previously, Unix was written in Assembly. Rewriting it in C changed everything:

Portability: Because C could be compiled for different architectures, Unix suddenly became a **portable operating system**. It could be moved to entirely different computer systems with minimal code modifications.

Design Goals: C successfully balanced high-level abstraction with low-level hardware control. It was fast, efficient, portable, and easy to write.

Step 6 : The Evolution of C Standards

As C grew in popularity, it needed formal rules to prevent different companies from fracturing the language.

Standard Version	Year	Key Highlights & Changes
K&R C	1978	Named after Brian Kernighan and Dennis Ritchie . They wrote a famous book, <i>The C Programming Language</i> . It served as the first standard reference for C.
ANSI C / C89	1989	The American National Standards Institute established the first official, unified standard rules for the language.
C90	1990	ISO (International Organization for Standardization) adopted ANSI C, turning it into the first globally accepted standard.
C99	1999	Introduced modern features, including support for single-line comments (//) alongside traditional block comments (/* */).
C11	2011	Focused on security and modernization. Added multithreading support and removed dangerous legacy functions.
C17	2018	A maintenance release. It introduced no major new features, focusing entirely on bug fixes and technical repairs for C11.
C23	2023/2024	The latest modern standard, introducing contemporary language improvements while strictly preserving C's signature simplicity and speed.

Important Syntax Clarifications & Legacy Fixes :-

The Assignment Operator (=)

In C, the = symbol denotes **assignment**, not mathematical equality. It copies the value from the right side and stores it inside the variable on the left.

`x = 10; // This puts the value 10 inside the variable x`

The Legacy Ambiguity of `x = -5;`

Before strict standardization, syntax interpretations varied wildly between different compilers:

Old Interpretation: `x = -5;` was often interpreted by early compilers as `x -= 5;` (which translates to `x = x - 5`).

Modern Standardized Interpretation: `x = -5;` explicitly means "assign the negative value 5 to the variable `x`".

The Removal of `gets()` in C11

The `gets()` function was officially deprecated and removed because it was deeply unsafe, possessing no built-in buffer limits. This frequently led to severe memory corruption and buffer overflow exploits.

✗ **Avoid (Unsafe):** `gets(name);`

✓ **Use Instead (Safe):** `fgets(name, 50, stdin);`

Basic Structure of a C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    statements;
```

```
}
```

Components:

1. Header File
2. Main Function
3. Braces
4. Statements

Part 1: Header File

#include <stdio.h>

here stdio.h stands for - **standard input output header file**

So by writing include <stdio.h> we Include Standard Input Output

Library It provides functions like:

printf()

scanf()

Think of it as bringing tools into your toolbox before starting work.

Part 2: Main Function

int main()

Every C program starts execution from main(). Think of it as:

Main Gate of the Program

The computer enters here first.

Part 3: Opening and Closing Braces

```
{  
}
```

Everything inside belongs to the main function.

Example:

```
int main()  
{  
    statements  
}
```

Part 4: Statements

Instructions given to the computer.

Example:

```
printf("Hello World");
```

Every statement ends with:

```
;
```

This semicolon acts like a full stop in English.

Part 2: Main Function

```
int main()
```

Every C program starts execution from main().

Think of it as:

Main Gate of the Program

The computer enters here first.

Part 3: Opening and Closing Braces

```
{  
}
```

Everything inside belongs to the main function.

Example:

```
int main()  
{  
    statements  
}
```

Part 4: Statements

Instructions given to the computer.

Example:

```
printf("Hello World");
```

Every statement ends with:

```
;
```

This semicolon acts like a full stop in English.